CCES – College of Computer Engineering and Sciences

Prince Sattam Bin Abdulaziz University

CS3401 – Algorithm Design and Analysis

Chapter 7

Greedy Algorithms

Academic Year 2026 – 2027

Dr. Khalil Chebil

Dr. Esraa

Chapter Overview

Introduction to Greedy Algorithms

Activity Selection Problem

Huffman Coding

Dijkstra's Shortest Path

Minimum Spanning Trees

Greedy vs. Dynamic Programming

Section 1

Introduction to Greedy Algorithms

What is a Greedy Algorithm?

Definition

A **greedy algorithm** builds a solution step by step, always making the *locally optimal choice* (the one that looks best at the current moment), without reconsidering past decisions.

Greedy vs. Dynamic Programming:

	Greedy	DP
Sub-problems	independent	overlapping
Choices	one per step	all options
Speed	fast (usually $\mathcal{O}(n \log n)$)	slower
Correctness	not always!	always correct

Greedy is not always correct

The change-making problem with coins $\{1, 3, 4\}$ and target 6:

- Greedy: $4 + 1 + 1 = 3$ coins $\leftarrow$ suboptimal
- DP: $3 + 3 = 2$ coins $\leftarrow$ optimal

When greedy works

When the problem has *greedy choice*

Section 2

Activity Selection Problem

Activity Selection

Problem

Given n activities each with start time s_i and finish time f_i, select the maximum number of *non-overlapping* activities.

Greedy strategy: Always pick the activity with the *earliest finish time* that does not conflict with the previously selected activity.

Algorithm:

```
1: Sort activities by finish time
2: Select activity 1 (earliest finish)
3: last ← 1
4: for i ← 2 to n
5:     if s_i ≥ f_last then
6:         Select activity i;  last ← i
7:     end if
8: end for
```

Complexity: $\mathcal{O}(n \log n)$ (sort) + $\mathcal{O}(n)$ (scan)

Example:

Selected: 1, 4, 8 (3 activities)

Proof of correctness: Exchange argument — replacing any selected activity with a conflicting one cannot improve the solution.

Section 3

Huffman Coding

Huffman Coding

Problem

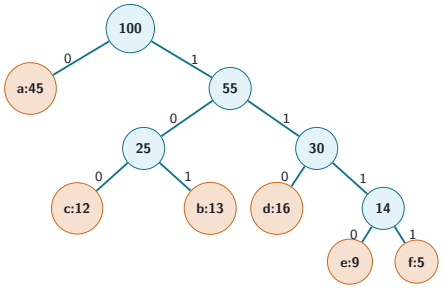
Given character frequencies, assign variable-length binary codes so that the *total encoding length* is minimised. More frequent characters get shorter codes.

Greedy algorithm:

1. Build a min-priority queue of leaf nodes (by frequency)
2. Repeat $n - 1$ times:
   - Extract two nodes with minimum frequency
   - Create internal node with sum of frequencies
   - Insert internal node back
3. The resulting tree is the Huffman tree

Complexity: $\mathcal{O}(n \log n)$

Example: Frequencies a:45, b:13, c:12, d:16, e:9, f:5



Section 4

Dijkstra's Shortest Path

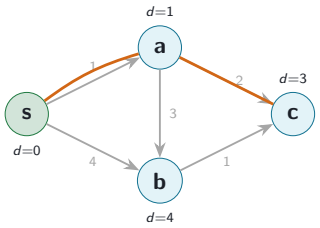
Dijkstra's Algorithm

Problem

Given a weighted graph with non-negative edge weights, find the *shortest paths* from a single source vertex s to all other vertices.

Algorithm

```
1: d[s] ← 0;  d[v] ← ∞ for v ≠ s
2: Priority queue Q ← all vertices
3: while Q not empty
4:     u ← vertex in Q with min d[u]
5:     remove u from Q
6:     for each neighbour v of u
7:         if d[u] + w(u, v) < d[v] then
8:             d[v] ← d[u] + w(u, v)
9:         end if
```



Shortest to c: s→a→c (cost 3)

Complexity: $\mathcal{O}((|V| + |E|) \log |V|)$ with binary heap.

Section 5

Minimum Spanning Trees

Minimum Spanning Tree — Problem

Definition

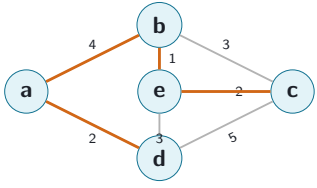
A **spanning tree** of a connected graph $G = (V, E)$ is a sub-graph that is a tree containing all $|V|$ vertices. A **minimum spanning tree (MST)** has the minimum total edge weight among all spanning trees.

Applications:

- Network design (cable layout, pipelines)
- Cluster analysis in machine learning
- Circuit board design (VLSI)
- Approximation algorithms for TSP

Two greedy algorithms:

1. **Prim's algorithm** — grow one tree, add nearest vertex
2. **Kruskal's algorithm** — add cheapest safe



MST cost = 2+1+2+4 = 9

Prim's Algorithm

Strategy: Start with any vertex; repeatedly add the *minimum-weight edge* connecting the MST so far to a vertex not yet in it.

Algorithm

1: $T \leftarrow \{v_0\}$, $E_T \leftarrow \emptyset$
2: **while** $|T| < n$
3: Find min-weight edge (u, v) with $u \in T$, $v \notin T$
4: $T \leftarrow T \cup \{v\}$; $E_T \leftarrow E_T \cup \{(u, v)\}$
5: **end while**
6: **return** (T, E_T)

Complexity

With adjacency matrix: $\Theta(n^2)$
With min-heap: $\mathcal{O}(|E| \log |V|)$

Why it is correct: *Cut property*: for any cut $(S, V \setminus S)$ of the graph, the minimum-weight edge crossing the cut is safe to add to the MST. (Proof by exchange argument.)

Related problem

Prim's algorithm is analogous to *Dijkstra's*; the only difference is the priority key: distance vs. edge weight.

Kruskal's Algorithm

Strategy: Sort all edges by weight; add an edge if it does not create a cycle (check with Union-Find).

Algorithm

1: Sort edges $e_1 \leq e_2 \leq \cdots \leq e_{|E|}$
2: Initialise Union-Find on V
3: $E_T \leftarrow \emptyset$
4: **for** $i \leftarrow 1$ **to** $|E|$
5: **if** $e_i = (u, v)$ and Find$(u) \neq$ Find(v) **then**
6: $E_T \leftarrow E_T \cup \{e_i\}$
7: Union(u, v)
8: **end if**
9: **end for**
10: **return** E_T

Complexity

Sorting: $\mathcal{O}(|E| \log |E|)$
Union-Find: $\mathcal{O}(|E| \cdot \alpha(|V|))$ (near-linear)
Total: $\mathcal{O}(|E| \log |E|)$

Prim's vs. Kruskal's:

	Prim	Kruskal	
Dense graphs	Better	Slower	Both
Sparse graphs	Equal	Better	
Data structure	Priority Q	Union-Find	

produce an MST with weight $= \sum_{e \in E_T} w(e)$.

Section 6

Greedy vs. Dynamic Programming

When to Use Which?

Problem	Greedy	DP
Activity selection	✓ optimal	Yes
Huffman coding	✓ optimal	Yes
MST (Prim, Kruskal)	✓ optimal	N/A
0-1 Knapsack	suboptimal	✓ optimal
Change-making (US)	✓ optimal	Yes
Change-making (gen.)	suboptimal	✓ optimal
Shortest path (dag)	✓ optimal	Yes
TSP	heuristic only	exact (slow)

Decision guide

1. Does the problem have *optimal substructure*?
   - No → neither works directly
2. Does it have *overlapping sub-problems*?
   - Yes → likely DP
   - No → likely D&C
3. Does the *greedy choice property* hold?
   - Yes → greedy (faster)
   - No → DP (always correct)

Chapter 7 Summary

Greedy algorithms studied:

Problem	Complexity
Activity selection	$\mathcal{O}(n \log n)$
Huffman coding	$\mathcal{O}(n \log n)$
Dijkstra's SSSP	$\mathcal{O}((\lvert V\rvert+\lvert E\rvert) \log \lvert V\rvert)$
Prim's MST	$\mathcal{O}(\lvert E\rvert \log \lvert V\rvert)$
Kruskal's MST	$\mathcal{O}(\lvert E\rvert \log \lvert E\rvert)$

Key: prove **correctness** using exchange argument or cut property before claiming a greedy algorithm is optimal.

Course Summary

You have now studied all major algorithm design paradigms:

- **Brute Force**
- **Decrease & Conquer**
- **Divide & Conquer**
- **Transform & Conquer**
- **Dynamic Programming**
- **Greedy Algorithms**

Next steps

NP-completeness, approximation algorithms, and randomised algorithms are advanced topics that build on this